⚡ CampusPulse — Project Overview & Master Architecture Guide

Version: 1.0.0 (Production)

Classification: Institutional Platform Documentation

Target Audience: Engineering Leads, System Administrators, Event Coordinators, Developers

Last Updated: September 2026

---

1. Executive Summary

CampusPulse is an enterprise-grade campus event discovery, dynamic survey, live engagement, and institutional reporting platform designed specifically for modern university ecosystems. Built on top of **Next.js 16 (Turbopack)**, **React 19**, and **MongoDB Atlas**, the platform consolidates disjointed college activities into a unified, secure, and mobile-ready web & native Android experience.

Traditional university events suffer from paper registration chaos, low feedback participation, untracked attendance, and fragmented media galleries. CampusPulse resolves these issues by delivering:

- Instant scannable digital boarding passes with encrypted QR tokens.
- Live public verification for security guards and event coordinators (`/events/verify/[token]`).
- Real-time interactive Q&A forums where attendees can upvote questions and speakers can pin answers.
- Google-Forms-style dynamic survey builders with automated NPS and sentiment analytics.
- Automated Executive Event Report generation with letterhead-ready PDF exports.
- Cross-platform support for Web, Mobile PWA, and Native Android (`CampusPulse.apk`).

---

2. High-Level Technology Stack

CampusPulse leverages modern web standards and high-reliability serverless paradigms:

Layer	Technology	Version	Purpose / Notes
Framework	Next.js (App Router)	`16.3.3`	Server Actions, Streaming SSR, Turbopack builder
Frontend UI	React	`19.2.8`	Concurrent rendering, Server & Client Components
Styling	Tailwind CSS + PostCSS	`v4.0.0`	Ultra-fast CSS compilation, custom dark palette
Database	MongoDB Atlas via Mongoose	`9.9.4`	Document-oriented storage, cached singleton pooling
Authentication	NextAuth.js	`^4.24.15`	JWT session strategy, credential auth, bcrypt encryption
Mobile Core	Capacitor CLI & Core	`^8.5.0`	Native Android WebView bridge, scheme isolation
Analytics Engine	Recharts	`^3.10.1`	Attendance trends, rating curves, category distribution
PDF Engine	`@react-pdf/renderer`	`^4.8.1`	Administrative letterhead reports & receipts
Validation	Zod	`^4.4.3`	Schema validation for user inputs and server actions
Animations	Framer Motion	`^13.1.1`	Micro-interactions, modal transitions, smooth fades
Icons	Lucide React	`^1.34.0`	Lightweight SVG icon library

3. System Architecture Diagram



```
|  +-------------------------------------------------------------------------+  |
+-----------------------------------------------------------------------------+
```

4. Complete Repository Directory Map

Below is a breakdown of every root file and folder in the CampusPulse repository:

```
campuspules/
├── .env.example              # Environment variables template for deployments & developers
├── .env.local                # Local active credentials (MongoDB, JWT secrets) - strictly gitignored
├── .firebaserc               # Firebase project mapping configuration (campuspulse-app-f6422)
├── .gitignore                # Production ignore rules (excludes build cache, node_modules, secrets)
├── AGENTS.md                 # Agent and developer guide conventions
├── CampusPulse.apk           # Compiled release Android binary ready for sideloading
├── Dockerfile                # Production multi-stage Docker build specification (Node 20 Alpine)
├── README.md                 # Project introduction, feature summary, and local run instructions
├── android/                  # Complete Native Android Studio project generated via Capacitor
├── apphosting.yaml           # Google Cloud / Firebase App Hosting compute and scaling configuration
├── capacitor.config.ts       # Capacitor native app ID, app name, and webDir routing config
├── docker-compose.yml        # Docker Compose service definition with uploads volume mounting
├── eslint.config.mjs         # ESLint 9 configuration with Next.js Core Web Vitals overrides
├── firebase.json             # Firebase Web Frameworks backend and hosting specification
├── netlify.toml              # Netlify 100% free serverless deployment config with security headers
├── next.config.ts            # Next.js 16 configuration with 50MB server action payload limits
├── package.json              # Node dependencies, scripts (dev, build, start, lint, firebase)
├── postcss.config.mjs        # PostCSS 4 configuration for Tailwind CSS compilation
├── tsconfig.json             # TypeScript compiler settings, path aliases (@/* -> ./src/*)
├── public/                   # Static public assets
│   ├── college-gate.jpg      # High-resolution campus gate hero banner
│   ├── favicon.ico           # Application brand favicon
│   ├── manifest.json         # Progressive Web App (PWA) manifest configuration
│   └── uploads/              # Local uploads directory for event banners and videos
├── scripts/                  # Administrative automation scripts
│   └── seed.js               # Comprehensive MongoDB Atlas database seeder
├── src/                      # Application source code
│   ├── actions/              # Server Actions for mutations and database operations
│   ├── app/                  # Next.js App Router (pages, layouts, API endpoints, sitemaps)
│   ├── components/           # Reusable React components (Layout, UI, Media, Providers)
│   ├── lib/                  # Core utilities (DB connection, Auth options, RBAC, Video helper)
│   ├── models/               # Mongoose data models and schemas
│   └── types/                # TypeScript interface definitions
└── docs/                     # Comprehensive A-to-Z modular documentation & PDF manuals
```

5. User Roles & Permission Matrices

CampusPulse utilizes strict **Role-Based Access Control (RBAC)** across all Server Actions and UI routes:

Feature / Action	Anonymous Visitor	Student Member	Event Coordinator / Admin
Browse Upcoming Events	Yes	Yes	Yes
RSVP / Obtain QR Pass	No (Redirects to Login)	Yes	Yes
Verify QR Passes at Door	Yes (Public /events/verify)	Yes	Yes
Submit Feedback Surveys	No	Yes	Yes
Post Questions in Live Q&A	No	Yes	Yes
Upvote Q&A Questions	No	Yes	Yes
Pin / Answer Q&A Questions	No	No	Yes
Create / Edit Events	No	No	Yes
Upload Media & Videos	No	No	Yes
Access Admin Dashboard	No	No	Yes
Download PDF Executive Reports	No	No	Yes
Manage Campus Settings	No	No	Yes

6. Security & Architectural Best Practices

1. **Password Encryption:** Passwords are never stored in plaintext. Passwords undergo bcrypt hashing with 10 salt rounds before persisting in MongoDB.
2. **Session Hardening:** All authenticated sessions use stateless, encrypted JSON Web Tokens (JWT) with a 30-day sliding expiry.
3. **Database Fault Tolerance:** Mongoose connection pooling is cached on the Node global object to eliminate connection starvation during serverless cold starts. Custom Google DNS resolvers prevent SRV lookup failures on corporate WiFi.
4. **Resilient Uploads:** Files are written to `/public/uploads` in local/docker environments. If deployed to a read-only serverless host, the system automatically falls back to Base64 Data URIs so file uploads never crash.